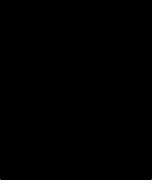


BarberShop Management Database

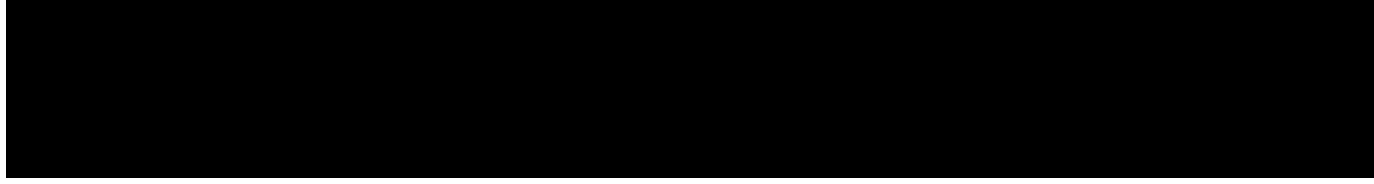
P3G1: 102480 - Rúben Pequeno, 134391 - Eduardo Assis

Tabela de Conteudos

01	Diagramas	Diagramas DER, ER Código DDL
02	Aplicação	UI + Video
03	Funcionalidades SQL	Queries, Stored Procedures, UDFs, Triggers
04	Conclusão	



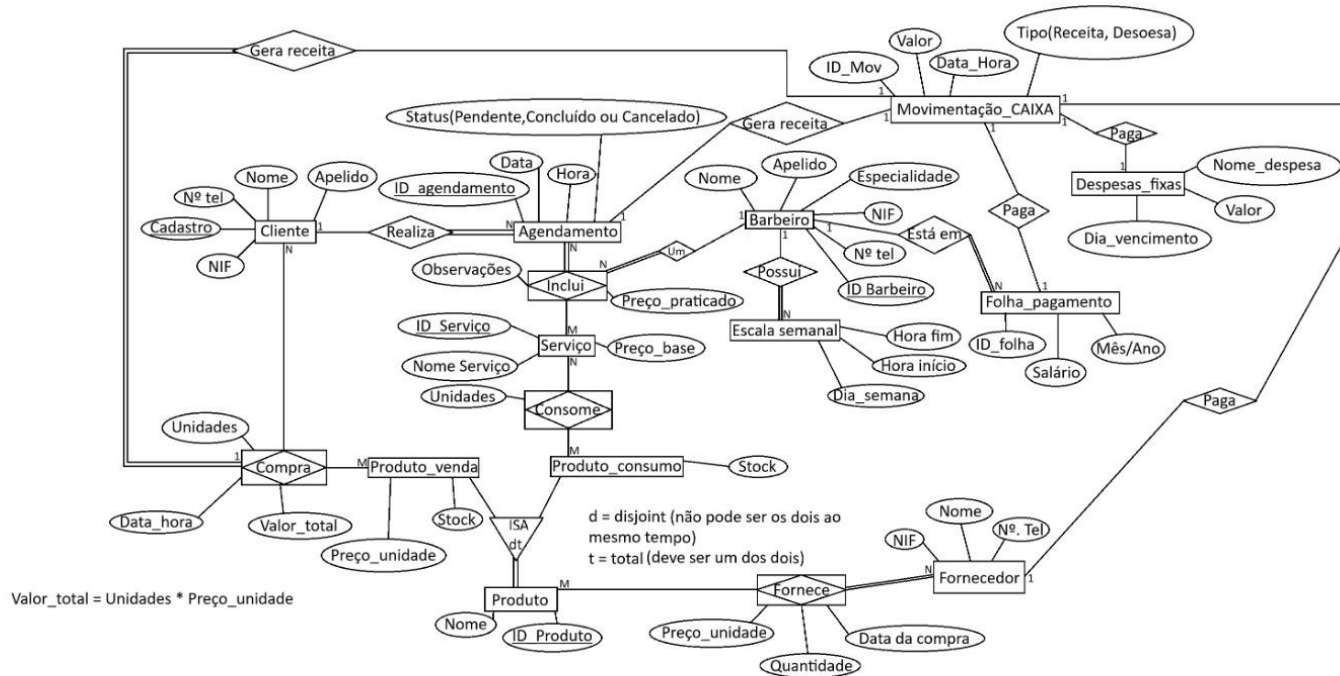
01



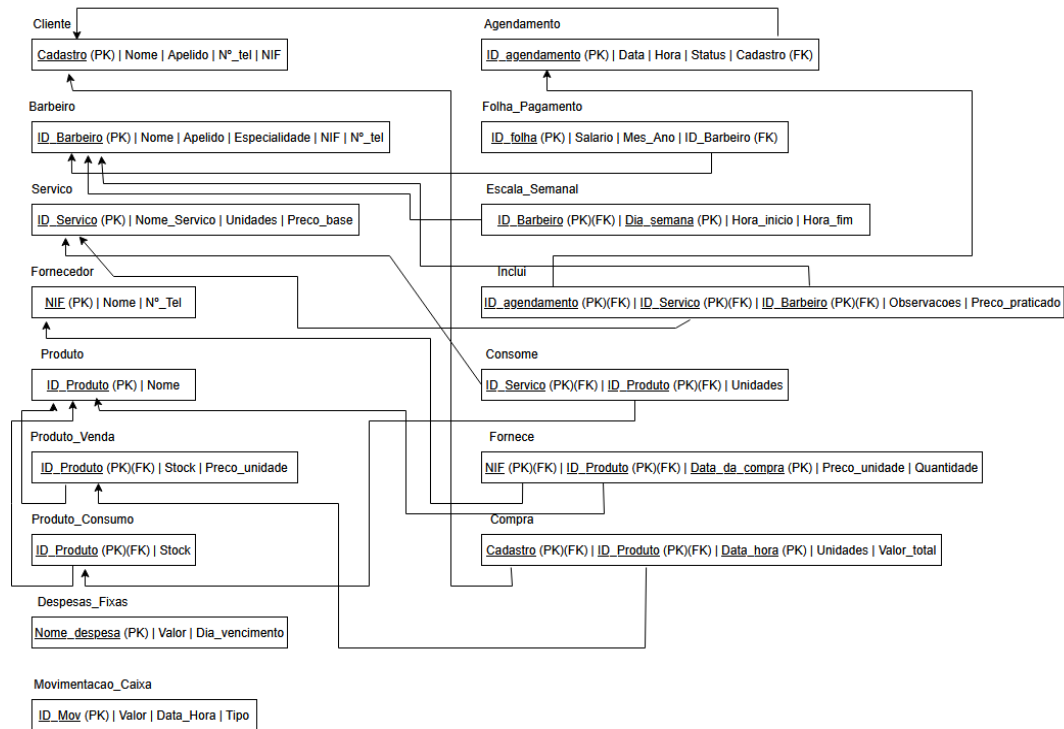
Diagramas



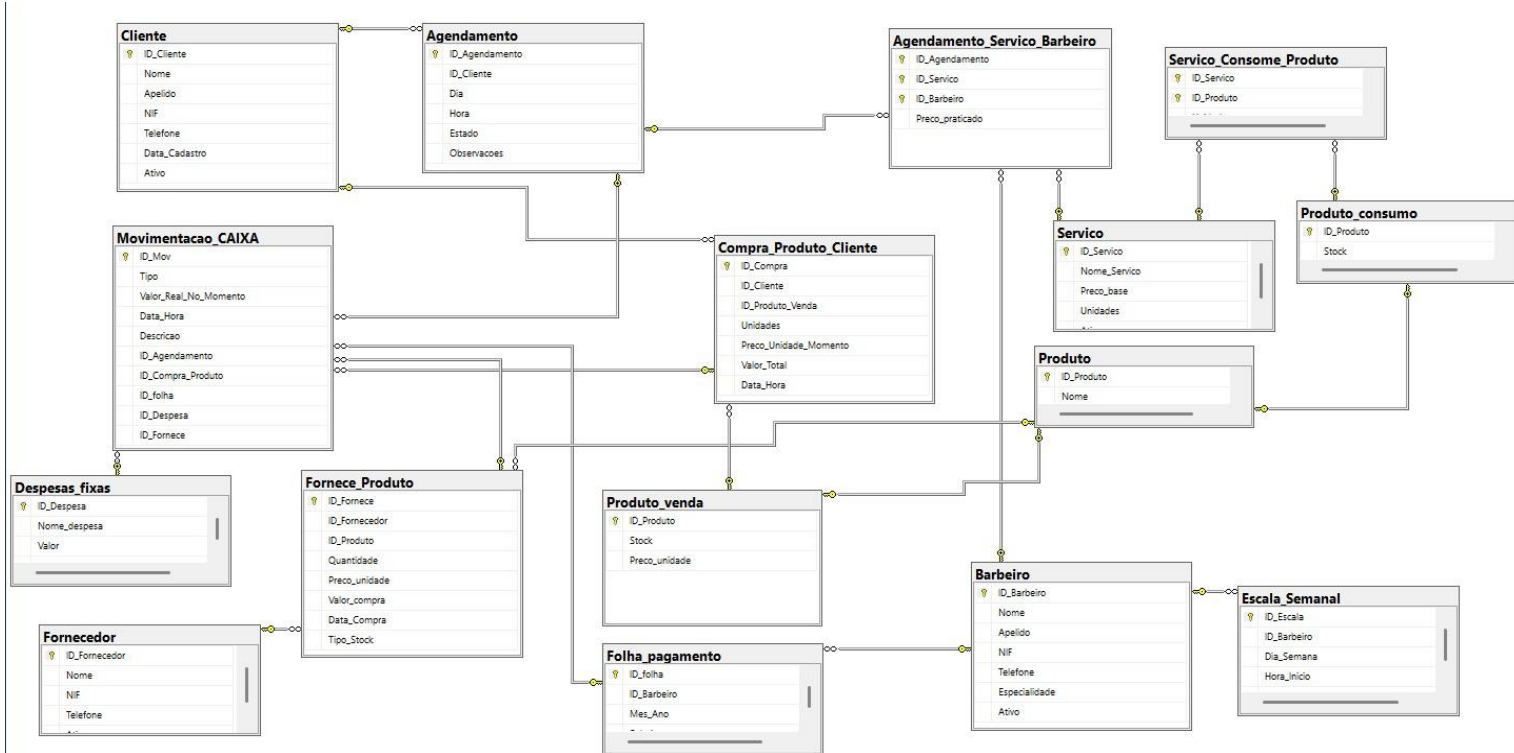
DER



ER



ER



Codigo DDL

Data Definition Language

Código DDL

-- Tabela: Cliente

```
CREATE TABLE Cliente (  
  ID_Cliente    INT IDENTITY(1,1) NOT NULL,  
  Nome          VARCHAR(50) NOT NULL,  
  Apelido       VARCHAR(50) NOT NULL,  
  NIF           CHAR(9)  NULL,  
  Telefone      VARCHAR(20) NULL,  
  Data_Cadastro DATE NULL DEFAULT GETDATE(),  
  Ativo         BIT      NOT NULL DEFAULT 1,  
  CONSTRAINT PK_Cliente PRIMARY KEY (ID_Cliente),  
  CONSTRAINT UQ_Cliente_NIF UNIQUE (NIF)  
);
```

-- Tabela: Barbeiro

```
CREATE TABLE Barbeiro (  
  ID_Barbeiro   INT      IDENTITY(1,1) NOT NULL,  
  Nome          VARCHAR(50) NOT NULL,  
  Apelido       VARCHAR(50) NOT NULL,  
  NIF           CHAR(9)  NULL,  
  Telefone      VARCHAR(20) NULL,  
  Especialidade VARCHAR(100) NULL,  
  Ativo         BIT      NOT NULL DEFAULT 1,  
  CONSTRAINT PK_Barbeiro PRIMARY KEY (ID_Barbeiro),  
  CONSTRAINT UQ_Barbeiro_NIF UNIQUE (NIF)  
);
```


Código DDL

```
-- Tabela: Fornecedor
CREATE TABLE Fornecedor (
  ID_Fornecedor INT      IDENTITY(1,1) NOT NULL,
  Nome          VARCHAR(100) NOT NULL,
  NIF           CHAR(9)   NULL,
  Telefone      CHAR(9)   NULL,
  Ativo         BIT       NOT NULL DEFAULT 1,
  CONSTRAINT PK_Fornecedor PRIMARY KEY (ID_Fornecedor),
  CONSTRAINT UQ_Fornecedor_NIF UNIQUE (NIF)
);
```

```
-- Tabela: Produto
CREATE TABLE Produto (
  ID_Produto INT      IDENTITY(1,1) NOT NULL,
  Nome        VARCHAR(100) NOT NULL,
  CONSTRAINT PK_Produto PRIMARY KEY (ID_Produto)
);
```

Código DDL

```
-- Tabela: Produto_consumo
CREATE TABLE Produto_consumo (
  ID_Produto  INT      NOT NULL,
  Stock       INT      NULL DEFAULT 0,
  CONSTRAINT PK_Produto_consumo PRIMARY KEY
(ID_Produto),
  CONSTRAINT FK_Produto_consumo_Produto
FOREIGN KEY (ID_Produto) REFERENCES Produto
(ID_Produto)
);
```

```
-- Tabela: Produto_venda
CREATE TABLE Produto_venda (
  ID_Produto  INT      NOT NULL,
  Stock       INT      NULL DEFAULT 0,
  Preco_unidade DECIMAL(10,2) NULL,
  CONSTRAINT PK_Produto_venda PRIMARY KEY
(ID_Produto),
  CONSTRAINT FK_Produto_venda_Produto FOREIGN KEY
(ID_Produto) REFERENCES Produto (ID_Produto)
);
```

Código DDL

-- Tabela: Servico

```
CREATE TABLE Servico (  
  ID_Servico    INT          IDENTITY(1,1) NOT NULL,  
  Nome_Servico  VARCHAR(100) NOT NULL,  
  Preco_base    DECIMAL(10,2) NOT NULL,  
  Unidades      INT          NULL DEFAULT 1,  
  Ativo         BIT          NOT NULL DEFAULT 1,  
  CONSTRAINT PK_Servico PRIMARY KEY (ID_Servico)  
);
```

-- Tabela: Servico_Consume_Produto

```
CREATE TABLE Servico_Consume_Produto (  
  ID_Servico    INT  NOT NULL,  
  ID_Produto     INT  NOT NULL,  
  Unidades       INT  NOT NULL,  
  CONSTRAINT PK_Servico_Consume_Produto PRIMARY  
KEY (ID_Servico, ID_Produto),  
  CONSTRAINT FK_SCP_Servico FOREIGN KEY (ID_Servico)  
REFERENCES Servico (ID_Servico),  
  CONSTRAINT FK_SCP_Produto FOREIGN KEY (ID_Produto)  
REFERENCES Produto_consumo (ID_Produto)  
);
```

Código DDL

```
-- Tabela: Agendamento
CREATE TABLE Agendamento (
  ID_Agendamento INT          IDENTITY(1,1) NOT NULL,
  ID_Cliente     INT          NOT NULL,
  Dia            DATE         NOT NULL,
  Hora           TIME(7)      NOT NULL,
  Estado         VARCHAR(20)  NULL DEFAULT
'Pendente',
  Observacoes   TEXT         NULL,
  CONSTRAINT PK_Agendamento PRIMARY KEY
(ID_Agendamento),
  CONSTRAINT FK_Agendamento_Cliente FOREIGN
KEY (ID_Cliente)
  REFERENCES Cliente (ID_Cliente),
  CONSTRAINT CHK_Estado_Agendamento CHECK (
  Estado IN ('Pendente', 'Concluído', 'Cancelado')
)
);
```

```
-- Tabela: Agendamento_Servico_Barbeiro
CREATE TABLE Agendamento_Servico_Barbeiro (
  ID_Agendamento INT          NOT NULL,
  ID_Servico      INT          NOT NULL,
  ID_Barbeiro     INT          NOT NULL,
  Preco_praticado DECIMAL(10,2) NULL,
  CONSTRAINT PK_Agendamento_Servico_Barbeiro PRIMARY
KEY (ID_Agendamento, ID_Servico, ID_Barbeiro),
  CONSTRAINT FK_ASB_Agendamento FOREIGN KEY
(ID_Agendamento)
  REFERENCES Agendamento (ID_Agendamento),
  CONSTRAINT FK_ASB_Servico FOREIGN KEY (ID_Servico)
  REFERENCES Servico (ID_Servico),
  CONSTRAINT FK_ASB_Barbeiro FOREIGN KEY
(ID_Barbeiro)
  REFERENCES Barbeiro (ID_Barbeiro)
);
```

Código DDL

-- Tabela: Compra_Produto_Cliente

```
CREATE TABLE Compra_Produto_Cliente (  
  ID_Compra      INT      IDENTITY(1,1) NOT NULL,  
  ID_Cliente     INT      NOT NULL,  
  ID_Produto_Venda INT      NOT NULL,  
  Unidades       INT      NOT NULL,  
  Preco_Unidade_Momento DECIMAL(10,2) NOT NULL,  
  Valor_Total    AS (Unidades *  
  Preco_Unidade_Momento), -- coluna calculada  
  Data_Hora      DATETIME  NULL DEFAULT  
  GETDATE(),  
  CONSTRAINT PK_Compra_Produto_Cliente PRIMARY KEY  
  (ID_Compra),  
  CONSTRAINT FK_CPC_Cliente FOREIGN KEY (ID_Cliente)  
  REFERENCES Cliente (ID_Cliente),  
  CONSTRAINT FK_CPC_Produto_venda FOREIGN KEY  
  (ID_Produto_Venda)  
  REFERENCES Produto_venda (ID_Produto)  
);
```

-- Tabela: Escala_Semanal

```
CREATE TABLE Escala_Semanal (  
  ID_Escala     INT      IDENTITY(1,1) NOT NULL,  
  ID_Barbeiro   INT      NOT NULL,  
  Dia_Semana    VARCHAR(20) NOT NULL,  
  Hora_Inicio   TIME(7)   NOT NULL,  
  Hora_Fim      TIME(7)   NOT NULL,  
  CONSTRAINT PK_Escala_Semanal PRIMARY KEY  
  (ID_Escala),  
  CONSTRAINT FK_Escala_Barbeiro FOREIGN KEY  
  (ID_Barbeiro)  
  REFERENCES Barbeiro (ID_Barbeiro),  
  CONSTRAINT CHK_Dia_Semana CHECK (  
    Dia_Semana IN ('Segunda-feira', 'Terça-feira', 'Quarta-feira',  
    'Quinta-feira', 'Sexta-feira', 'Sábado', 'Domingo')  
  )  
);
```

Código DDL

-- Tabela: Folha_pagamento

```
CREATE TABLE Folha_pagamento (  
  ID_folha      INT          IDENTITY(1,1) NOT NULL,  
  ID_Barbeiro   INT          NOT NULL,  
  Mes_Ano       DATE         NOT NULL,  
  Salario       DECIMAL(10,2) NOT NULL,  
  CONSTRAINT PK_Folha_pagamento PRIMARY KEY  
    (ID_folha),  
  CONSTRAINT FK_Folha_Barbeiro FOREIGN KEY  
    (ID_Barbeiro)  
    REFERENCES Barbeiro (ID_Barbeiro)  
);
```

-- Tabela: Despesas_fixas

```
CREATE TABLE Despesas_fixas (  
  ID_Despesa    INT          IDENTITY(1,1) NOT NULL,  
  Nome_despesa  NVARCHAR(100) NOT NULL,  
  Valor         DECIMAL(10,2) NOT NULL,  
  Dia_vencimento DATE        NOT NULL,  
  CONSTRAINT PK_Despesas_fixas PRIMARY KEY  
    (ID_Despesa)  
);
```

Código DDL

```
-- Tabela: Fornece_Produto
CREATE TABLE Fornece_Produto (
  ID_Fornece INT IDENTITY(1,1) NOT NULL,
  ID_Fornecedor INT NOT NULL,
  ID_Produto INT NOT NULL,
  Quantidade INT NOT NULL,
  Preco_unidade DECIMAL(10,2) NOT NULL,
  Valor_compra AS (Quantidade * Preco_unidade), -- coluna
calculada
  Data_Compra DATE NULL DEFAULT GETDATE(),
  Tipo_Stock VARCHAR(10) NOT NULL,
  CONSTRAINT PK_Fornece_Produto PRIMARY KEY
(ID_Fornece),
  CONSTRAINT FK_FP_Fornecedor FOREIGN KEY
(ID_Fornecedor)
REFERENCES Fornecedor (ID_Fornecedor),
  CONSTRAINT FK_FP_Produto FOREIGN KEY (ID_Produto)
REFERENCES Produto (ID_Produto),
  CONSTRAINT CHK_Tipo_Stock CHECK (
  Tipo_Stock IN ('Venda', 'Consumo')
)
);
```

```
-- Tabela: Movimentacao_CAIXA CREATE TABLE
Movimentacao_CAIXA ( ID_Mov INT IDENTITY(1,1) NOT NULL,
Tipo VARCHAR(20) NOT NULL,
Valor_Real_No_Momento DECIMAL(10,2) NOT NULL,
Data_Hora DATETIME NULL DEFAULT GETDATE(),
Descricao VARCHAR(255) NULL, ID_Agendamento INT NULL,
ID_Compra_Produto INT NULL,
ID_folha INT NULL, ID_Despesa INT NULL, ID_Fornece INT NULL,
CONSTRAINT PK_Movimentacao_CAIXA PRIMARY KEY (ID_Mov),
CONSTRAINT FK_MOV_Agendamento FOREIGN KEY
(ID_Agendamento) REFERENCES Agendamento (ID_Agendamento),
CONSTRAINT FK_MOV_Compra FOREIGN KEY
(ID_Compra_Produto) REFERENCES Compra_Produto_Cliente
(ID_Compra),
CONSTRAINT FK_MOV_Folha FOREIGN KEY (ID_folha)
REFERENCES Folha_pagamento (ID_folha),
CONSTRAINT FK_MOV_Despesa FOREIGN KEY (ID_Despesa)
REFERENCES Despesas_fixas (ID_Despesa),
CONSTRAINT FK_MOV_Fornece FOREIGN KEY (ID_Fornece)
REFERENCES Fornece_Produto (ID_Fornece),
CONSTRAINT CHK_Tipo_Movimentacao CHECK ( Tipo IN ('Receita',
'Despesa') ) );
```



02

Aplicação



Screenshots

FormMenu

CAIXA

Despesas fixas

Estoque de produtos

Cientes

Vencimentos

Fornecedores

Barbeiros

Vendas

Servicos

Novo Agendamento

Concluir Agendamento

Cancelar Agendamento

	Nome Cliente	Nome Barbeiro	Data	Horário	Valor Total	Esta
►	Jorge	Jenny	26/05/2026	13:41:32	40,00	Conc
	Livia	Antony	26/05/2026	14:30:37	15,00	Conc
	Livia	Antony	26/05/2026	14:55:53.4966001	25,00	Conc
	Livia	Jenny	26/05/2026	14:55:53.4966001	15,00	Conc
	Eduardo	Jenny	25/05/2026	11:07:45.0024763	15,00	Conc

FormAgendamento

Cliente

Serviço

Data/Hora

27/05/2026

18:52:44

Barbeiro

Estado do agendamento

Pendente

Preço praticado

Observações

Adicionar Serviço ao Agendamento

Gravar Agendamento

	ID_Serviço	Nome_Serviço	ID_Barbeiro	Nome_Ba
*				

Screenshots

FormMovimentacao

Filtrar por categoria

Tipo

Nome

Saldo (€) 1 989,51 €

Filtrar por data

Início 27/05/2026

Fim 27/05/2026

Filtrar

	Tipo	Valor (€)	Data/Hora	Descrição
▶	Receita	35,00	18/05/2026 20:50	Serviço(s) prestado(s) n...
	Receita	20,00	18/05/2026 20:50	Serviço(s) prestado(s) n...
	Despesa	20,00	18/05/2026 20:50	Estomo por Cancelamen...
	Receita	20,00	18/05/2026 20:51	Serviço(s) prestado(s) n...
	Despesa	35,00	18/05/2026 20:52	Estomo por Cancelamen...
	Receita	35,00	18/05/2026 20:52	Serviço(s) prestado(s) n...
	Despesa	35,00	20/05/2026 13:16	Estomo por Cancelamen...
	Despesa	20,00	20/05/2026 13:16	Estomo por Cancelamen...

Nome Jenny

Telefone 234153452

Apelido Williams

Especialidade Corte Feminino

NIF 292032941

Escala

Início 18:57:31

Fim 18:57:31

☐ Segunda-feira

☐ Terça-feira

☐ Quarta-feira

☐ Quinta-feira

Limpar

Excluir

Gravar

Barbeiro

	ID_Barbeiro	Nome	Ap
▶	8	Jenny	Wil
	9	Jack	Da
	12	Antony	Sta
	15	Bradd	Pitt
	16	Kristen	Ste

Escala

	Dia_Semana	Hora_Inicio	Hora_Fim
▶	Segunda-feira	08:00:00	16:00:00
	Terça-feira	08:00:00	16:00:00
	Quarta-feira	08:00:00	16:00:00
	Quinta-feira	08:00:00	16:00:00
	Sexta-feira	08:00:00	16:00:00

Screenshots

FormStock

Atualizar Estoques

Fornecedor: Assistec Unidade(s): 0

Produto: Gel de cabelo Preço de compra por unidade (€):

Tipo: Preço de venda por unidade (€):

Gravar

Estoque para consumo				Estoque para vendas			
	ID_Produto	Nome	Stoc		ID_Produto	Nome	Stoc
▶	2	Gel de cabelo	19	▶	2	Gel de cabelo	18
	3	Shampoo	12		3	Shampoo	10
	4	Tinta de cabelo	4		4	Tinta de cabelo	14
	5	Lâmina de barbear	8		8	Perfume	10

FormServico

Corte - 15,00 €
Barba - 10,00 €
Tingimento - 25,00 €

Nome do Serviço:

Preço (€):

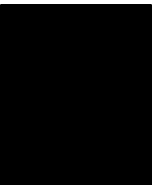
tempo (min):

Produto consumido (Tipo/unidades): 0

Adicionar Produto Excluir Gravar

	ID_Produto	Nome_Produto	Unidades
*			

Video



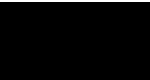
03



Funcionalidades SQL

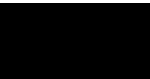


Queries



Query Example 1

Filtrar histórico de movimentações



Query Example 2

Ver saldo atual

Exemplo Querie 1

Filtrar histórico de movimentações

```
SELECT
```

```
Tipo, Valor_Real_No_Momento AS [Valor (€)],
```

```
Data_Hora AS [Data/Hora],
```

```
Descricao AS [Descrição]
```

```
FROM dbo.Movimentacao_CAIXA
```

```
WHERE Data_Hora >= @ini AND Data_Hora <= @fim
```

Exemplo Querie 2

Ver saldo atual

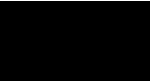
```
SELECT
```

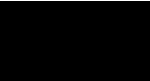
```
ISNULL(SUM(CASE WHEN Tipo = 'Receita' THEN Valor_Real_No_Momento ELSE 0 END), 0) -
```

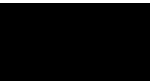
```
ISNULL(SUM(CASE WHEN Tipo = 'Despesa' THEN Valor_Real_No_Momento ELSE 0 END), 0)
```

```
AS SaldoAtual FROM dbo.Movimentacao_CAIXA
```


Outras funcionalidades

 Stored Procedures

 Triggers

 UDFs

Exemplo Stored Procedure 1

```
CREATE PROCEDURE sp_Dashboard_GetAgendamentos
AS
BEGIN
    SET NOCOUNT ON;

    SELECT
        A.ID_Agendamento,
        C.Nome AS [Nome Cliente],
        B.Nome AS [Nome Barbeiro],
        A.Dia AS [Data],
        A.Hora AS [Horário],
        SUM(ASB.Preco_praticado) AS [Valor Total],
        A.Estado AS [Estado]
    FROM dbo.Agendamento A
    INNER JOIN dbo.Cliente C ON A.ID_Cliente = C.ID_Cliente
    INNER JOIN dbo.Agendamento_Servico_Barbeiro ASB ON A.ID_Agendamento = ASB.ID_Agendamento
    INNER JOIN dbo.Barbeiro B ON ASB.ID_Barbeiro = B.ID_Barbeiro
    GROUP BY A.ID_Agendamento, C.Nome, B.Nome, A.Dia, A.Hora, A.Estado
    ORDER BY A.Dia DESC, A.Hora ASC;
END
```

Exemplo Stored Procedure 2

```
CREATE PROCEDURE sp_Agendamento_UpdateEstado
    @ID_Agendamento INT,
    @NovoEstado NVARCHAR(50)
AS
BEGIN
    SET NOCOUNT ON;

    BEGIN TRANSACTION;

    BEGIN TRY
        -- verifica se o agendamento existe
        IF NOT EXISTS (SELECT 1 FROM dbo.Agendamento WHERE ID_Agendamento = @ID_Agendamento)
        BEGIN
            RAISERROR('Agendamento não encontrado.', 16, 1);
            ROLLBACK TRANSACTION;
            RETURN;
        END

        -- Atualização do estado
        UPDATE dbo.Agendamento
        SET Estado = @NovoEstado
        WHERE ID_Agendamento = @ID_Agendamento;

        COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
        ROLLBACK TRANSACTION;
        THROW; -- manda erro para a aplicação
    END CATCH
END
```

Exemplo Trigger

```
CREATE TRIGGER TRG_Consume_Produto_No_Servico
ON dbo.Agendamento
AFTER UPDATE
AS
BEGIN
    SET NOCOUNT ON;

    IF EXISTS (
        SELECT 1
        FROM inserted i
        INNER JOIN deleted d ON i.ID_Agendamento = d.ID_Agendamento
        WHERE i.Estado = 'Concluído' AND d.Estado <> 'Concluído'
    )
    BEGIN
        UPDATE PC
        SET PC.Stock = PC.Stock - SCP.Unidades
        FROM dbo.Produto_consumo PC
        INNER JOIN dbo.Servico_Consume_Produto SCP ON PC.ID_Produto = SCP.ID_Produto
        INNER JOIN dbo.Agendamento_Servico_Barbeiro ASB ON SCP.ID_Servico = ASB.ID_Servico
        INNER JOIN inserted i ON ASB.ID_Agendamento = i.ID_Agendamento
        INNER JOIN deleted d ON i.ID_Agendamento = d.ID_Agendamento
        WHERE i.Estado = 'Concluído' AND d.Estado <> 'Concluído';
    END
END;
```

Exemplo UDF

```
CREATE FUNCTION udf_TotalVendasCliente (@ID_Cliente INT)
RETURNS DECIMAL(10,2)
AS
BEGIN
    DECLARE @Total DECIMAL(10,2);

    SELECT @Total = ISNULL(SUM(Unidades * Preco_Unidade_Momento), 0)
    FROM dbo.Compra_Produto_Cliente
    WHERE ID_Cliente = @ID_Cliente;

    RETURN @Total;
END;
```



04

Conclusão



Conclusão

O que aprendemos

Conceitos de modelação e várias funcionalidades SQL

Melhorias

Normalização, índices para optimização, ...

Alguma questão?

Obrigado!

